Experiment 10

Longest Increasing Subsequence (LIS) Using Dynamic Programming

Aim

To write a Java program to find the Longest Increasing Subsequence (LIS) of a given array using Dynamic Programming, display its length and actual subsequence, and compare the $O(n^2)$Dynamic Programming approach with the $O(n \log n)$binary search approach.

Theory

The Longest Increasing Subsequence (LIS) problem involves finding the longest subsequence of an array in which the elements appear in their original order and are strictly increasing. The elements of a subsequence do not need to be adjacent.

The $O(n^2)$Dynamic Programming approach maintains an array dp[], where dp[i] represents the length of the longest increasing subsequence ending at index i. Each element is compared with the previous elements to calculate the maximum length. A predecessor array is used to reconstruct the actual subsequence.

The $O(n \log n)$approach uses binary search to maintain the smallest possible ending element for increasing subsequences of different lengths. It is faster, but reconstructing the actual subsequence requires tracking predecessor indices and the indices of the best ending elements.

- **Time complexity:** $O(n^2)$for the basic Dynamic Programming approach and $O(n \log n)$for the binary search approach.
- **Space complexity:** $O(n)$for both approaches.

Algorithm

1. Read the number of elements and array values from the user.
2. Initialize the DP array with 1, since each element forms a subsequence of length 1.
3. Compare each element with all previous elements and update the DP values when an increasing subsequence is found.
4. Track predecessor indices to reconstruct the actual LIS.
5. Display the LIS length and the subsequence obtained using Dynamic Programming.
6. Apply the binary search approach to calculate the LIS length in $O(n \log n)$time.
7. Display the result of both approaches and compare their time complexities.

Program / Code

```
import java.util.*;

public class Main {

    static void lisDP(int[] arr) {
        int n = arr.length;
        int[] dp = new int[n];
        int[] prev = new int[n];

        Arrays.fill(dp, 1);
        Arrays.fill(prev, -1);

        int maxLen = 1, end = 0;

        for (int i = 1; i < n; i++) {
```

```
            for (int j = 0; j < i; j++) {
                if (arr[j] < arr[i] && dp[j] + 1 > dp[i]) {
                    dp[i] = dp[j] + 1;
                    prev[i] = j;
                }
            }

            if (dp[i] > maxLen) {
                maxLen = dp[i];
                end = i;
            }
        }

        List<Integer> lis = new ArrayList<>();
        while (end != -1) {
            lis.add(arr[end]);
            end = prev[end];
        }
        Collections.reverse(lis);

        System.out.println("\nO(n^2) Dynamic Programming:");
        System.out.println("LIS Length: " + maxLen);
        System.out.println("LIS: " + lis);
    }

    static int lisBinarySearch(int[] arr) {
        int[] tails = new int[arr.length];
        int size = 0;

        for (int x : arr) {
            int left = 0, right = size;

            while (left < right) {
                int mid = left + (right - left) / 2;

                if (tails[mid] < x)
                    left = mid + 1;
                else
                    right = mid;
            }

            tails[left] = x;

            if (left == size)
                size++;
        }

        return size;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter number of elements: ");
        int n = sc.nextInt();

        if (n <= 0) {
            System.out.println("Array size must be positive.");
            sc.close();
            return;
        }

        int[] arr = new int[n];
```

```
        System.out.println("Enter array elements:");
        for (int i = 0; i < n; i++)
            arr[i] = sc.nextInt();

        lisDP(arr);

        System.out.println("\nO(n log n) Binary Search:");
        System.out.println("LIS Length: " + lisBinarySearch(arr));

        System.out.println("\nComplexity Comparison:");
        System.out.println("DP Approach: O(n^2)");
        System.out.println("Binary Search Approach: O(n log n)");

        sc.close();
    }
}
```

Input

```
Enter number of elements: 8
Enter array elements: 10 9 2 5 3 7 101 18
```

Output

```
O(n^2) Dynamic Programming:
LIS Length: 4
LIS: [2, 5, 7, 101]

O(n log n) Binary Search:
LIS Length: 4

Complexity Comparison:
DP Approach: O(n^2)
Binary Search Approach: O(n log n)
```

Conclusion

The Longest Increasing Subsequence problem was successfully implemented in Java using Dynamic Programming and binary search. The program accepted an array from the user and displayed the length and actual subsequence using the $O(n^2)$approach. It also calculated the LIS length using the $O(n \log n)$approach. Both methods produced the same LIS length for the sample input. The experiment demonstrated how Dynamic Programming solves the problem by storing intermediate results, while binary search improves efficiency for larger arrays.

Learning Outcomes

- Understood the Longest Increasing Subsequence problem and its applications.
- Learned to use Dynamic Programming to calculate the LIS length and reconstruct the subsequence.
- Understood how binary search reduces the time complexity to $O(n \log n)$.
- Compared the efficiency of the $O(n^2)$and $O(n \log n)$approaches.
- Analysed the time and space complexity of both algorithms.
- Learned how subsequence problems can be solved efficiently using algorithmic optimization.